Important Questions - Machine Learning Algorithms

Question 1

Explain how cross-validation, particularly K-fold cross-validation, helps in improving model performance and reduces the chances of overfitting. Provide examples of how it compares to using a simple train-test split. Draw K-fold CV flow diagram.

Answer:

What is Cross-Validation?

Cross-validation is a statistical technique used to evaluate machine learning models by partitioning the data into subsets, training the model on some subsets, and validating it on the remaining subsets.

K-Fold Cross-Validation Process:

Step 1: Data Splitting

- Divide the dataset into K equal-sized folds (typically K=5 or K=10)
- Each fold serves as a validation set once while the remaining K-1 folds serve as training data

Step 2: Model Training and Validation

```
For i = 1 to K:
    - Use fold i as validation set
    - Use remaining K-1 folds as training set
    - Train model on training set
    - Evaluate model on validation set
    - Store performance metric
```

Step 3: Final Performance Calculation

- Average the performance metrics across all K iterations
- Calculate standard deviation to understand variance

K-Fold CV Flow Diagram:

```
Original Dataset
├── Fold 1 │ Fold 2 │ Fold 3 │ Fold 4 │ Fold 5 │

Iteration 1: [Test] │ [Train] │ [Train] │ [Train] │ [Train] │
Iteration 2: [Train] │ [Test] │ [Train] │ [Train] │ [Train] │
Iteration 3: [Train] │ [Train] │ [Test] │ [Train] │ [Train] │
Iteration 4: [Train] │ [Train] │ [Train] │ [Test] │ [Train] │
Iteration 5: [Train] │ [Train] │ [Train] │ [Train] │ [Test] │
```

```
Final Score = Average of all 5 iteration scores
```

How K-Fold CV Reduces Overfitting:

1. **Multiple Validation Sets**: Uses different portions of data for validation, reducing bias
2. **Better Generalization Assessment**: Tests model performance on various data subsets
3. **Variance Reduction**: Averages multiple performance estimates
4. **Robust Model Selection**: Helps choose models that perform consistently across different data splits

Comparison: K-Fold CV vs Simple Train-Test Split

Aspect	Train-Test Split	K-Fold CV
Data Usage	~70% training, ~30% testing	~90% training, ~10% testing (per fold)
Evaluation Robustness	Single evaluation	K evaluations averaged
Overfitting Detection	Limited	Better detection
Computational Cost	Low	K times higher
Variance in Results	High	Lower

Example Scenarios:

Train-Test Split Example:

```
# Dataset: 1000 samples
# Train: 700 samples, Test: 300 samples
# Single accuracy: 85%
# Problem: What if the 300 test samples are not representative?
```

K-Fold CV Example:

```
# Dataset: 1000 samples, K=5
# Each fold: 200 samples for testing, 800 for training
# Results: [84%, 87%, 83%, 86%, 85%]
# Average: 85% ± 1.58%
# More reliable estimate with confidence interval
```

When to Use K-Fold CV:

- Small to medium datasets
- Model selection and hyperparameter tuning
- When you need robust performance estimates
- Research and competition settings

Question 2

You have built a binary classifier to predict whether emails are spam or not spam. After testing your classifier on a dataset of 500 emails, you obtain the following confusion matrix:

N=500	Actual Not Spam	Actual Spam
Predicted Not Spam	350	20
Predicted Spam	30	100

a) Calculate the overall accuracy of the classifier
b) Calculate the precision, recall, and F1-score for each class.
c) Analyze the classifier's performance based on these metrics and discuss which class the model struggles with the most.

Answer:

Understanding the Confusion Matrix:

```
                    Actual
                Not Spam    Spam
Predicted  Not Spam  350      20    (370 total predicted as Not Spam)
           Spam       30     100    (130 total predicted as Spam)
           Total     380     120    (500 total emails)
```

Key Terms:

- **True Positive (TP)**: Correctly predicted Spam = 100
- **True Negative (TN)**: Correctly predicted Not Spam = 350
- **False Positive (FP)**: Incorrectly predicted as Spam = 30
- **False Negative (FN)**: Incorrectly predicted as Not Spam = 20

a) Overall Accuracy Calculation:

Formula:

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

Calculation:

```
Accuracy = (100 + 350) / (100 + 350 + 30 + 20)
Accuracy = 450 / 500
Accuracy = 0.90 or 90%
```

b) Precision, Recall, and F1-Score for Each Class:

For SPAM Class (Positive Class):

Precision (Spam):

```
Precision = TP / (TP + FP) = 100 / (100 + 30) = 100/130 = 0.769 or 76.9%
```

Recall (Spam):

```
Recall = TP / (TP + FN) = 100 / (100 + 20) = 100/120 = 0.833 or 83.3%
```

F1-Score (Spam):

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
F1 = 2 × (0.769 × 0.833) / (0.769 + 0.833)
F1 = 2 × 0.641 / 1.602 = 1.282 / 1.602 = 0.800 or 80.0%
```

For NOT SPAM Class (Negative Class):

Precision (Not Spam):

```
Precision = TN / (TN + FN) = 350 / (350 + 20) = 350/370 = 0.946 or 94.6%
```

Recall (Not Spam):

```
Recall = TN / (TN + FP) = 350 / (350 + 30) = 350/380 = 0.921 or 92.1%
```

F1-Score (Not Spam):

```
F1 = 2 × (0.946 × 0.921) / (0.946 + 0.921)
F1 = 2 × 0.871 / 1.867 = 1.742 / 1.867 = 0.933 or 93.3%
```

Summary Table:

Class	Precision	Recall	F1-Score
Spam	76.9%	83.3%	80.0%
Not Spam	94.6%	92.1%	93.3%

c) Performance Analysis:

Overall Performance:

- **Accuracy: 90%** - Good overall performance
- The classifier performs significantly better on "Not Spam" emails

Class-wise Analysis:

Spam Detection Issues:

1. **Lower Precision (76.9%)**: Out of emails predicted as spam, only 76.9% are actually spam
    - **Impact**: 23.1% of legitimate emails are incorrectly marked as spam (False Positives)
    - **Business Impact**: Users might miss important emails
2. **Moderate Recall (83.3%)**: The model catches 83.3% of actual spam emails
    - **Impact**: 16.7% of spam emails go to inbox (False Negatives)
    - **Business Impact**: Some spam emails reach users

Not Spam Detection:

- **High Precision (94.6%)**: Very reliable when predicting "Not Spam"
- **High Recall (92.1%)**: Catches most legitimate emails correctly

Which Class Struggles Most?

The SPAM class struggles the most due to:

1. **Higher Error Rates**:
    - 30 False Positives (legitimate emails marked as spam)
    - 20 False Negatives (spam emails marked as legitimate)
2. **Lower Performance Metrics**: All metrics (precision, recall, F1) are lower for spam
3. **Class Imbalance Effect**:
    - Not Spam: 380 samples (76%)
    - Spam: 120 samples (24%)
    - The model is biased toward predicting the majority class

Recommendations for Improvement:

1. **Collect more spam examples** to balance the dataset
2. **Adjust classification threshold** to improve spam recall
3. **Use techniques like SMOTE** for handling class imbalance
4. **Feature engineering** to better distinguish spam characteristics
5. **Cost-sensitive learning** to penalize missing spam more heavily

---

Question 3

i. **Explain Linear regression with suitable examples**

What is Linear Regression?

Linear regression is a statistical method used to model the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a linear equation to the observed data.

Mathematical Foundation:

Simple Linear Regression (One feature):

```
y = β₀ + β₁x + ε
```

Where:

- y = dependent variable (target)
- x = independent variable (feature)
- β_0 = y-intercept (bias)
- β_1 = slope (weight)
- ε = error term

Multiple Linear Regression:

```
y = β₀ + β₁x₁ + β₂x₂ + ... + βₙxₙ + ε
```

How Linear Regression Works:

Step 1: Data Collection

- Gather data with input features and target values

Step 2: Model Fitting

- Find the best line that minimizes the sum of squared errors
- Use methods like Ordinary Least Squares (OLS)

Step 3: Parameter Estimation

```
β₁ = Σ[(xᵢ − x̄)(yᵢ − ȳ)] / Σ[(xᵢ − x̄)²]
β₀ = ȳ − β₁x̄
```

Step 4: Prediction

- Use the fitted line to predict new values

Example 1: House Price Prediction

Dataset:

House Size (sq ft)	Price ($000)
1000	200
1500	300
2000	400
2500	500
3000	600

Step-by-step Calculation:

```
x̄ = (1000+1500+2000+2500+3000)/5 = 2000
ȳ = (200+300+400+500+600)/5 = 400

β₁ = Σ[(xᵢ−x̄)(yᵢ−ȳ)] / Σ[(xᵢ−x̄)²]
   = [(−1000)(−200) + (−500)(−100) + (0)(0) + (500)(100) + (1000)(200)] /
[1000² + 500² + 0² + 500² + 1000²]
   = [200000 + 50000 + 0 + 50000 + 200000] / [1000000 + 250000 + 0 +
250000 + 1000000]
   = 500000 / 2500000 = 0.2

β₀ = 400 − 0.2(2000) = 0
```

Final Model: Price = 0.2 × Size **Interpretation:** Each additional square foot increases price by $200

Example 2: Student Grade Prediction

```
Grade = 50 + 0.8 × Hours_Studied
```

- If a student studies 10 hours: Grade = 50 + 0.8(10) = 58
- If a student studies 25 hours: Grade = 50 + 0.8(25) = 70

Assumptions of Linear Regression:

1. **Linearity**: Relationship between variables is linear
2. **Independence**: Observations are independent
3. **Homoscedasticity**: Constant variance of errors
4. **Normality**: Errors are normally distributed

---

ii. **Explain in detail about logistic regression classifier, advantages and applications with suitable example?**

What is Logistic Regression?

Logistic regression is a statistical method used for binary classification problems. Unlike linear regression that predicts continuous values, logistic regression predicts the probability of an instance belonging to a particular class.

Mathematical Foundation:

Sigmoid Function:

```
σ(z) = 1 / (1 + e^(-z))
where z = β₀ + β₁x₁ + β₂x₂ + ... + βₙxₙ
```

Logistic Regression Equation:

```
P(y=1|x) = 1 / (1 + e^(-(β₀ + β₁x₁ + β₂x₂ + ... + βₙxₙ)))
```

How Logistic Regression Works:

Step 1: Linear Combination

- Compute $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$

Step 2: Sigmoid Transformation

- Apply sigmoid function to get probability: $P = \sigma(z)$

Step 3: Classification Decision

- If $P \geq 0.5$, classify as Class 1
- If $P < 0.5$, classify as Class 0

Step 4: Parameter Estimation

- Use Maximum Likelihood Estimation (MLE)
- Minimize log-loss function

Example: Email Spam Detection

Features:

- x_1 = Number of exclamation marks
- x_2 = Number of capital letters
- x_3 = Presence of word "FREE" (1 if present, 0 if not)

Sample Data:

Email	Exclamation	Capital Letters	Word "FREE"	Spam (y)
1	5	20	1	1

Email	Exclamation	Capital Letters	Word "FREE"	Spam (y)
2	1	5	0	0
3	8	35	1	1
4	0	3	0	0

Trained Model (example):

```
z = -2.5 + 0.3×Exclamation + 0.1×Capital + 2.0×FREE
P(Spam) = 1 / (1 + e^(-z))
```

Prediction for New Email:

- Exclamation = 3, Capital = 15, FREE = 1
- z = -2.5 + 0.3(3) + 0.1(15) + 2.0(1) = -2.5 + 0.9 + 1.5 + 2.0 = 1.9
- P(Spam) = 1 / (1 + e^(-1.9)) = 1 / (1 + 0.15) = 0.87 or 87%
- **Classification: SPAM** (since 0.87 > 0.5)

Advantages of Logistic Regression:

1. **Probabilistic Output**: Provides probability estimates, not just classifications
2. **No Distribution Assumptions**: Doesn't assume normal distribution of features
3. **Less Overfitting**: With low-dimensional data, less prone to overfitting
4. **Fast Training**: Computationally efficient
5. **Interpretable**: Coefficients show feature importance and direction
6. **No Tuning Required**: Fewer hyperparameters to tune
7. **Linear Decision Boundary**: Creates clear, interpretable decision boundaries

Applications:

1. Medical Diagnosis

```
P(Disease) = f(symptoms, age, medical_history)
Example: P(Heart Disease) = f(chest_pain, age, cholesterol,
blood_pressure)
```

2. Marketing

```
P(Purchase) = f(income, age, previous_purchases, website_visits)
```

3. Finance

```
P(Loan Default) = f(credit_score, income, debt_ratio, employment_history)
```

4. Quality Control

```
P(Defective Product) = f(temperature, pressure, material_quality,
machine_age)
```

Limitations:

1. **Linear Decision Boundary**: Cannot handle complex non-linear relationships
2. **Feature Scaling**: Sensitive to feature scaling
3. **Outliers**: Can be affected by outliers
4. **Independence Assumption**: Assumes features are independent

Multiclass Extension:

- **One-vs-Rest (OvR)**: Train multiple binary classifiers
- **Multinomial Logistic Regression**: Direct multiclass approach using softmax function

---

Question 4

(i) **Explain simple linear regression using an example dataset, along with a detailed breakdown of the mathematical calculations involved in deriving the equation?**

Example Dataset: Ice Cream Sales vs Temperature

Day	Temperature (°C) (x)	Ice Cream Sales (units) (y)
1	20	100
2	25	150
3	30	200
4	35	250
5	40	300

Mathematical Derivation Step-by-Step:

Goal: Find the line $y = \beta_0 + \beta_1 x$ that best fits the data

Step 1: Calculate Means

```
x̄ = (20 + 25 + 30 + 35 + 40) / 5 = 150 / 5 = 30°C
ȳ = (100 + 150 + 200 + 250 + 300) / 5 = 1000 / 5 = 200 units
```

Step 2: Calculate Deviations

Day	x	y	$(x - \bar{x})$	$(y - \bar{y})$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$
1	20	100	-10	-100	1000	100
2	25	150	-5	-50	250	25
3	30	200	0	0	0	0
4	35	250	5	50	250	25
5	40	300	10	100	1000	100
Sum					**2500**	**250**

Step 3: Calculate Slope (β_1)

```
β₁ = Σ[(xᵢ − x̄)(yᵢ − ȳ)] / Σ[(xᵢ − x̄)²]
β₁ = 2500 / 250 = 10 units/°C
```

Step 4: Calculate Intercept (β_0)

```
β₀ = ȳ − β₁x̄
β₀ = 200 − 10(30) = 200 − 300 = −100 units
```

Step 5: Final Regression Equation

```
y = −100 + 10x
```

Interpretation:

- **Intercept (-100)**: Theoretical sales when temperature is 0°C (negative indicates extrapolation beyond data range)
- **Slope (10)**: For each 1°C increase in temperature, ice cream sales increase by 10 units

Step 6: Making Predictions

- At 28°C: y = -100 + 10(28) = 180 units
- At 42°C: y = -100 + 10(42) = 320 units

Verification using Normal Equations:

Matrix Form: $X'X\beta = X'y$

```
X = [1  20]    y = [100]
    [1  25]        [150]
    [1  30]        [200]
    [1  35]        [250]
    [1  40]        [300]

X'X = [5   150]    X'y = [1000]
      [150 4750]         [32500]

β = (X'X)⁻¹X'y = [−100]
                  [10]
```

(ii) **What are the evaluation metrics that are used for regression in Machine Learning? Explain Mean Absolute Error (MAE) and Mean Squared Error (MSE)?**

Regression Evaluation Metrics Overview:

Regression metrics measure how well our model predicts continuous values by comparing predicted values with actual values.

1. Mean Absolute Error (MAE)

Formula:

```
MAE = (1/n) × Σ|yᵢ − ŷᵢ|
```

Where:

- n = number of observations
- y_i = actual value
- $\hat{y}_i$ = predicted value

Characteristics:

- **Units**: Same as the target variable
- **Range**: [0, ∞), where 0 is perfect
- **Robustness**: Less sensitive to outliers
- **Interpretation**: Average absolute difference between predictions and actual values

Example Calculation:

```
Actual (y) = [100, 150, 200, 250, 300]
Predicted (ŷ) = [95, 160, 190, 255, 290]
|y − ŷ| = [5, 10, 10, 5, 10]
```

```
MAE = (5 + 10 + 10 + 5 + 10) / 5 = 40 / 5 = 8 units
```

Interpretation: On average, our predictions are off by 8 units.

2. Mean Squared Error (MSE)

Formula:

```
MSE = (1/n) × Σ(yᵢ − ŷᵢ)²
```

Characteristics:

- **Units**: Square of the target variable units
- **Range**: [0, ∞), where 0 is perfect
- **Sensitivity**: More sensitive to outliers (due to squaring)
- **Interpretation**: Average squared difference between predictions and actual values

Example Calculation:

Actual (y)	Predicted (ŷ)	(y - ŷ)²
100	95	25
150	160	100
200	190	100
250	255	25
300	290	100

```
MSE = (25 + 100 + 100 + 25 + 100) / 5 = 350 / 5 = 70 units²
```

Other Important Regression Metrics:

3. Root Mean Squared Error (RMSE)

```
RMSE = √MSE = √70 = 8.37 units
```

- Same units as target variable
- More interpretable than MSE

4. R-squared (Coefficient of Determination)

```
R² = 1 - (SSres / SStot)
where SSres = Σ(yᵢ - ŷᵢ)²
      SStot = Σ(yᵢ - ȳ)²
```

- Range: [0, 1] for good models
- Indicates proportion of variance explained

5. Mean Absolute Percentage Error (MAPE)

```
MAPE = (100/n) × Σ|((yᵢ - ŷᵢ)/yᵢ)|
```

- Expressed as percentage
- Scale-independent

MAE vs MSE Comparison:

Aspect	MAE	MSE
Outlier Sensitivity	Low	High
Mathematical Properties	Not differentiable at 0	Differentiable everywhere
Optimization	Harder to optimize	Easier to optimize
Interpretation	More intuitive	Less intuitive
Units	Same as target	Square of target units

When to Use Which:

Use MAE when:

- You want to treat all errors equally
- Outliers should not dominate the metric
- You need intuitive interpretation

Use MSE when:

- Large errors are particularly undesirable
- You're using gradient-based optimization
- Mathematical convenience is important

(iii) **Explain the concept of Overfitting and underfitting?**

What is Overfitting?

Definition: Overfitting occurs when a model learns the training data too well, including noise and random fluctuations, resulting in poor generalization to new, unseen data.

Characteristics:

- High accuracy on training data
- Poor accuracy on validation/test data
- Model is too complex for the underlying pattern
- Memorizes rather than learns general patterns

What is Underfitting?

Definition: Underfitting occurs when a model is too simple to capture the underlying pattern in the data, resulting in poor performance on both training and test data.

Characteristics:

- Low accuracy on training data
- Low accuracy on validation/test data
- Model is too simple for the underlying pattern
- High bias, low variance

Visual Representation:

```
Training Error vs Model Complexity

Error
  ↑
  |     Underfitting    |  Sweet Spot  |    Overfitting
  |                     |              |
  |        \            |              |      /
  |         \           |              |     /
  |          \          |              |    /
  |           \____     |              |   /
  |                \____|______________|__/  ← Validation Error
  |                     |              |
  |  ___________________|______________|_____→ Training Error
  |                     |              |
  └─────────────────────|______________|_____→
    Low              Optimal         High     Model Complexity
```

Examples:

Example 1: Polynomial Regression

Dataset: Temperature vs Ice Cream Sales

- True relationship: Linear (y = 10x - 100)

Underfitting (Degree 0 - Constant):

```
Model: y = 200 (horizontal line)
Training Error: High
Test Error: High
Problem: Too simple to capture linear trend
```

Good Fit (Degree 1 - Linear):

```
Model: y = 10x - 100
Training Error: Low
Test Error: Low
Result: Captures true relationship
```

Overfitting (Degree 10 - High-order polynomial):

```
Model: y = ax¹⁰ + bx⁹ + ... + cx + d
Training Error: Very Low (near 0)
Test Error: Very High
Problem: Fits noise in training data
```

Bias-Variance Tradeoff:

Bias: Error due to overly simplistic assumptions **Variance:** Error due to sensitivity to small fluctuations

Model State	Bias	Variance	Total Error
Underfitting	High	Low	High
Good Fit	Low	Low	Low
Overfitting	Low	High	High

Detecting Overfitting and Underfitting:

Learning Curves Analysis:

```
# Pseudo-code for learning curves
train_scores = []
val_scores = []

for training_size in [0.1, 0.2, ..., 1.0]:
    model.fit(X_train[:training_size], y_train[:training_size])
    train_score = model.score(X_train[:training_size],
y_train[:training_size])
    val_score = model.score(X_val, y_val)
    train_scores.append(train_score)
    val_scores.append(val_score)
```

Interpretation:

- **Gap between curves**: Indicates overfitting
- **Both curves low**: Indicates underfitting
- **Converging curves**: Indicates good fit

Solutions:

For Overfitting:

1. **Regularization**: Add penalty terms (L1/L2)
2. **Cross-validation**: Use k-fold CV for model selection
3. **Early stopping**: Stop training when validation error increases
4. **Reduce model complexity**: Fewer parameters/features
5. **More training data**: Helps model generalize better
6. **Dropout**: For neural networks
7. **Feature selection**: Remove irrelevant features

For Underfitting:

1. **Increase model complexity**: More parameters/layers
2. **Feature engineering**: Create more relevant features
3. **Reduce regularization**: Less penalty on complexity
4. **Train longer**: More epochs/iterations
5. **Ensemble methods**: Combine multiple simple models

Practical Example - Polynomial Features:

```
# Linear model (may underfit)
y = β₀ + β₁x

# Quadratic model (may be just right)
y = β₀ + β₁x + β₂x²

# High-degree polynomial (may overfit)
y = β₀ + β₁x + β₂x² + ... + β₁₀x¹⁰
```

Model Selection Strategy:

1. Start with simple model
2. Gradually increase complexity
3. Use validation set to monitor performance
4. Choose model with best validation performance
5. Confirm with test set

Question 5

(i) What is Logistic Regression? How you will convert the line equation into Sigmoid curve?

What is Logistic Regression?

Logistic Regression is a statistical method used for binary classification problems. It predicts the probability that an instance belongs to a particular class (usually labeled as 1 or 0, True or False, Yes or No).

Key Differences from Linear Regression:

- **Output**: Probabilities (0 to 1) instead of continuous values
- **Function**: Uses sigmoid function instead of linear function
- **Purpose**: Classification instead of regression

Converting Linear Equation to Sigmoid Curve:

Step 1: Start with Linear Equation

```
z = β₀ + β₁x₁ + β₂x₂ + ... + βₙxₙ
```

This is the linear combination of features (also called logits).

Step 2: Apply Sigmoid Function

```
σ(z) = 1 / (1 + e^(-z))
```

Where σ(z) represents the probability P(y=1|x).

Step 3: Complete Logistic Regression Equation

```
P(y=1|x) = 1 / (1 + e^(-(β₀ + β₁x₁ + β₂x₂ + ... + βₙxₙ)))
```

Mathematical Transformation Process:

Problem with Linear Regression for Classification:

- Linear function: $y = \beta_0 + \beta_1 x$ can produce values < 0 or > 1
- We need probabilities bounded between 0 and 1

Solution - Sigmoid Function Properties:

```
When z → +∞: σ(z) → 1
When z → -∞: σ(z) → 0
When z = 0: σ(z) = 0.5
```

Step-by-Step Derivation:

1. **Odds Ratio:**

```
Odds = P(y=1) / P(y=0) = P(y=1) / (1 - P(y=1))
```

2. **Log Odds (Logits):**

```
log(Odds) = log(P(y=1) / (1 - P(y=1))) = β₀ + β₁x
```

3. **Solve for P(y=1):**

```
P(y=1) / (1 - P(y=1)) = e^(β₀ + β₁x)
P(y=1) = (1 - P(y=1)) × e^(β₀ + β₁x)
P(y=1) = e^(β₀ + β₁x) - P(y=1) × e^(β₀ + β₁x)
P(y=1) + P(y=1) × e^(β₀ + β₁x) = e^(β₀ + β₁x)
P(y=1)(1 + e^(β₀ + β₁x)) = e^(β₀ + β₁x)
P(y=1) = e^(β₀ + β₁x) / (1 + e^(β₀ + β₁x))
P(y=1) = 1 / (1 + e^(-β₀ - β₁x))
```

Sigmoid Curve Characteristics:

```
P(y=1)
   ↑
1.0|           -------- (Asymptote)
   |         /
   |        /
0.5|-------/----------→ Decision Boundary
   |      /
   |     /
0.0|--------    (Asymptote)
   └──────────────────→ z
        0
```

Properties:

- **S-shaped curve**: Smooth transition from 0 to 1
- **Monotonic**: Always increasing
- **Symmetric**: Around point (0, 0.5)
- **Asymptotic**: Approaches but never reaches 0 or 1

Example: Student Pass/Fail Prediction

Linear Model (Wrong Approach):

```
Pass = 0.4 + 0.1 × Hours_Studied
Problems:
- At 0 hours: 40% (reasonable)
- At 10 hours: 140% (impossible!)
- At -5 hours: -10% (impossible!)
```

Logistic Model (Correct Approach):

```
z = -2 + 0.5 × Hours_Studied
P(Pass) = 1 / (1 + e^(-(-2 + 0.5 × Hours_Studied)))

Hours | z    | P(Pass)
------|------|--------
0     | -2   | 0.12 (12%)
2     | -1   | 0.27 (27%)
4     | 0    | 0.50 (50%)
6     | 1    | 0.73 (73%)
8     | 2    | 0.88 (88%)
10    | 3    | 0.95 (95%)
```

---

a) **How you will handle more than two class using Logistic regression? Explain with example?**

Multiclass Classification Strategies:

1. One-vs-Rest (OvR) / One-vs-All (OvA)

Concept: Train K binary classifiers, where K is the number of classes. Each classifier distinguishes one class from all others.

Algorithm:

```
For each class i:
    Create binary problem: Class i vs All other classes
    Train logistic regression classifier

For prediction:
    Run all K classifiers
    Choose class with highest probability
```

Example: Email Classification (3 classes)

- Classes: Spam, Personal, Work

Classifier 1: Spam vs (Personal + Work)

```
P₁(Spam | email) = σ(β₀₁ + β₁₁x₁ + β₂₁x₂ + ...)
```

Classifier 2: Personal vs (Spam + Work)

```
P₂(Personal | email) = σ(β₀₂ + β₁₂x₁ + β₂₂x₂ + ...)
```

Classifier 3: Work vs (Spam + Personal)

```
P₃(Work | email) = σ(β₀₃ + β₁₃x₁ + β₂₃x₂ + ...)
```

Prediction Example:

```
New Email Features: x₁=5, x₂=10, x₃=1

Classifier 1: P₁(Spam) = 0.3
Classifier 2: P₂(Personal) = 0.8
Classifier 3: P₃(Work) = 0.2

Prediction: Personal (highest probability)
```

2. Multinomial Logistic Regression (Softmax)

Concept: Direct extension of logistic regression using softmax function.

Softmax Function:

```
P(y = k | x) = e^(z_k) / Σ_{j=1}^K e^(z_j)

where z_k = β₀ₖ + β₁ₖx₁ + β₂ₖx₂ + ... + βₙₖxₙ
```

Properties:

- All probabilities sum to 1: $\Sigma_k P(y=k|x) = 1$
- Each probability is between 0 and 1

Example: Iris Flower Classification

Classes: Setosa, Versicolor, Virginica **Features:** Sepal Length (x_1), Sepal Width (x_2), Petal Length (x_3), Petal Width (x_4)

Model Equations:

```
z₁ = β₀₁ + β₁₁x₁ + β₂₁x₂ + β₃₁x₃ + β₄₁x₄  (Setosa)
z₂ = β₀₂ + β₁₂x₁ + β₂₂x₂ + β₃₂x₃ + β₄₂x₄  (Versicolor)
z₃ = β₀₃ + β₁₃x₁ + β₂₃x₂ + β₃₃x₃ + β₄₃x₄  (Virginica)
```

Probability Calculation:

```
P(Setosa) = e^z₁ / (e^z₁ + e^z₂ + e^z₃)
P(Versicolor) = e^z₂ / (e^z₁ + e^z₂ + e^z₃)
P(Virginica) = e^z₃ / (e^z₁ + e^z₂ + e^z₃)
```

Numerical Example:

```
Sample: x₁=5.1, x₂=3.5, x₃=1.4, x₄=0.2

Calculated z values:
z₁ = 2.5 (Setosa)
z₂ = -1.0 (Versicolor)
z₃ = -0.8 (Virginica)

Exponentials:
e^z₁ = e^2.5 = 12.18
e^z₂ = e^(-1.0) = 0.37
e^z₃ = e^(-0.8) = 0.45

Sum = 12.18 + 0.37 + 0.45 = 13.00

Probabilities:
P(Setosa) = 12.18/13.00 = 0.937 (93.7%)
P(Versicolor) = 0.37/13.00 = 0.028 (2.8%)
P(Virginica) = 0.45/13.00 = 0.035 (3.5%)

Prediction: Setosa
```

Comparison: OvR vs Multinomial

Aspect	One-vs-Rest	Multinomial
Number of Models	K models	1 model
Training Time	K × Binary training time	Direct multiclass training
Probability Interpretation	Not guaranteed to sum to 1	Always sum to 1
Decision Boundary	K linear boundaries	Direct multiclass boundaries
Implementation	Simpler	More complex

What is Regularization and explain different types of it?

What is Regularization?

Definition: Regularization is a technique used to prevent overfitting by adding a penalty term to the loss function, which constrains or shrinks the model parameters.

Purpose:

- Reduce model complexity
- Prevent overfitting
- Improve generalization
- Handle multicollinearity

Loss Function with Regularization:

General Form:

```
Total Loss = Original Loss + λ × Penalty Term

where λ (lambda) is the regularization parameter
```

Types of Regularization:

1. L1 Regularization (Lasso - Least Absolute Shrinkage and Selection Operator)

Penalty Term:

```
L1 Penalty = λ Σ_{j=1}^{n} |β_j|
```

Complete Loss Function:

```
L1_Loss = MSE + λ Σ_{j=1}^{n} |β_j|
```

Characteristics:

- **Feature Selection**: Can set coefficients exactly to zero
- **Sparse Solutions**: Creates sparse models
- **Robust**: Less sensitive to outliers
- **Non-differentiable**: At $\beta = 0$

Example:

```
Original model: y = 2x₁ + 0.1x₂ + 0.05x₃ + 1.8x₄
After L1 (λ=0.1): y = 1.8x₁ + 0x₂ + 0x₃ + 1.6x₄

Features x₂ and x₃ are eliminated (coefficients = 0)
```

2. L2 Regularization (Ridge)

Penalty Term:

```
L2 Penalty = λ Σⱼ₌₁ⁿ βⱼ²
```

Complete Loss Function:

```
L2_Loss = MSE + λ Σⱼ₌₁ⁿ βⱼ²
```

Characteristics:

- **Shrinkage**: Reduces coefficients toward zero (but not exactly zero)
- **Smooth Solutions**: All features retained with smaller weights
- **Handles Multicollinearity**: Better when features are correlated
- **Differentiable**: Everywhere

Example:

```
Original model: y = 2x₁ + 0.1x₂ + 0.05x₃ + 1.8x₄
After L2 (λ=0.1): y = 1.6x₁ + 0.08x₂ + 0.04x₃ + 1.4x₄

All features retained but coefficients are smaller
```

3. Elastic Net (L1 + L2 Combination)

Penalty Term:

```
Elastic Net Penalty = λ₁ Σⱼ₌₁ⁿ |βⱼ| + λ₂ Σⱼ₌₁ⁿ βⱼ²
```

Alternative Form:

```
Penalty = λ [α Σⱼ₌₁ⁿ |βⱼ| + (1−α) Σⱼ₌₁ⁿ βⱼ²]

where α ∈ [0,1] controls the mix:
α = 1: Pure L1 (Lasso)
```

```
α = 0: Pure L2 (Ridge)
α = 0.5: Equal mix
```

Characteristics:

- **Best of Both**: Combines feature selection (L1) and shrinkage (L2)
- **Group Selection**: Selects groups of correlated features
- **Stable**: More stable than pure L1

Comparison of Regularization Types:

Aspect	L1 (Lasso)	L2 (Ridge)	Elastic Net
Feature Selection	Yes (sets to 0)	No (shrinks)	Yes (selective)
Handles Multicollinearity	Poorly	Well	Well
Solution Stability	Less stable	More stable	Most stable
Interpretability	High	Medium	High
Computational Cost	Higher	Lower	Highest

Practical Example: Housing Price Prediction

Dataset Features:

- x_1: Square footage
- x_2: Number of bedrooms
- x_3: Number of bathrooms
- x_4: Age of house
- x_5: Distance to city center

Without Regularization:

```
Price = 50000 + 100×sqft + 5000×bedrooms + 8000×bathrooms − 500×age −
2000×distance
```

Problem: May overfit with large coefficients

With L1 Regularization (λ=1000):

```
Price = 45000 + 90×sqft + 4000×bedrooms + 0×bathrooms − 400×age +
0×distance
```

Result: Bathrooms and distance features eliminated

With L2 Regularization (λ=1000):

```
Price = 48000 + 85×sqft + 4200×bedrooms + 3000×bathrooms − 420×age −
800×distance
```

Result: All features retained but coefficients shrunk

Choosing Regularization Parameter (λ):

Methods:

1. **Cross-Validation**: Try different λ values and choose best CV score
2. **Grid Search**: Systematic search over λ values
3. **Learning Curves**: Plot training/validation error vs λ

Typical λ values to try:

```
λ ∈ [0.001, 0.01, 0.1, 1, 10, 100, 1000]
```

Effect of λ:

- **λ = 0**: No regularization (may overfit)
- **Small λ**: Light regularization
- **Large λ**: Heavy regularization (may underfit)

Question 6

(i) **Explain in detail about Cross validation and its types?**

What is Cross-Validation?

Definition: Cross-validation is a resampling technique used to evaluate machine learning models by partitioning the data into subsets, training the model on some subsets, and validating it on the remaining subsets.

Purpose:

- Assess model performance more reliably
- Detect overfitting and underfitting
- Compare different models objectively
- Tune hyperparameters effectively

Types of Cross-Validation:

1. K-Fold Cross-Validation

Process:

1. Divide dataset into K equal-sized folds
2. For each fold i (i = 1 to K):
    - Use fold i as validation set
    - Use remaining K-1 folds as training set
    - Train and evaluate model
3. Average the K performance scores

Example: 5-Fold CV with 1000 samples

```
Fold 1: [Test: 1-200]   [Train: 201-1000]
Fold 2: [Train: 1-200]  [Test: 201-400]   [Train: 401-1000]
Fold 3: [Train: 1-400]  [Test: 401-600]   [Train: 601-1000]
Fold 4: [Train: 1-600]  [Test: 601-800]   [Train: 801-1000]
Fold 5: [Train: 1-800]  [Test: 801-1000]

Results: [85%, 87%, 83%, 86%, 84%]
Average: 85% ± 1.58%
```

Advantages:

- Each sample used for both training and validation
- Reduces variance in performance estimate
- Good for small to medium datasets

Disadvantages:

- Computationally expensive (K times slower)
- May not preserve class distribution

2. Stratified K-Fold Cross-Validation

Purpose: Maintains the same proportion of samples from each class in each fold.

Example: Binary Classification with 60% Class 0, 40% Class 1

```
Original Distribution: 600 Class 0, 400 Class 1

Each fold maintains 60:40 ratio:
Fold 1: 120 Class 0, 80 Class 1
Fold 2: 120 Class 0, 80 Class 1
Fold 3: 120 Class 0, 80 Class 1
Fold 4: 120 Class 0, 80 Class 1
Fold 5: 120 Class 0, 80 Class 1
```

When to Use:

- Imbalanced datasets
- Classification problems

- When class distribution is important

3. Leave-One-Out Cross-Validation (LOOCV)

Process:

- K = n (number of samples)
- Each iteration uses n-1 samples for training, 1 for validation
- Repeat n times

Example: Dataset with 100 samples

```
Iteration 1: Train on samples 2-100, Test on sample 1
Iteration 2: Train on samples 1,3-100, Test on sample 2
...
Iteration 100: Train on samples 1-99, Test on sample 100

Average 100 individual results
```

Advantages:

- Maximum use of data for training
- No randomness in train/test splits
- Good for very small datasets

Disadvantages:

- Computationally very expensive
- High variance in estimates
- May overestimate performance

4. Time Series Cross-Validation (Forward Chaining)

Purpose: Respects temporal order in time series data.

Process:

```
Time: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Fold 1: Train[1,2,3]     Test[4]
Fold 2: Train[1,2,3,4]   Test[5]
Fold 3: Train[1,2,3,4,5] Test[6]
Fold 4: Train[1,2,3,4,5,6] Test[7]
...
```

Key Rule: Never use future data to predict past data.

5. Group K-Fold Cross-Validation

Purpose: Ensures that samples from the same group don't appear in both training and validation sets.

Example: Medical Data

```
Groups: Patients A, B, C, D, E (each with multiple samples)

Fold 1: Train[B,C,D,E] Test[A]
Fold 2: Train[A,C,D,E] Test[B]
Fold 3: Train[A,B,D,E] Test[C]
Fold 4: Train[A,B,C,E] Test[D]
Fold 5: Train[A,B,C,D] Test[E]
```

6. Repeated K-Fold Cross-Validation

Process:

1. Perform K-fold CV
2. Repeat with different random splits
3. Average all results

Example: 5-Fold CV repeated 3 times

```
Run 1: [85%, 87%, 83%, 86%, 84%] → Avg: 85%
Run 2: [86%, 84%, 85%, 87%, 83%] → Avg: 85%
Run 3: [84%, 86%, 84%, 85%, 86%] → Avg: 85%

Final Average: 85%
More reliable estimate with lower variance
```

Choosing the Right Cross-Validation:

Data Type	Recommended CV	Reason
Small Dataset (<1000)	LOOCV or 10-fold	Maximize training data
Large Dataset (>10000)	5-fold or 3-fold	Computational efficiency
Imbalanced Classes	Stratified K-fold	Preserve class distribution
Time Series	Time Series CV	Respect temporal order
Grouped Data	Group K-fold	Prevent data leakage

---

a) **What are the different performance metrics of a Classification model? Explain Confusion matrix table with example?**

Confusion Matrix

Definition: A confusion matrix is a table used to evaluate the performance of a classification model. It shows the actual vs predicted classifications.

Binary Classification Confusion Matrix:

```
                    Actual
                 Positive  Negative
Predicted Positive   TP      FP     | Predicted Positive
          Negative   FN      TN     | Predicted Negative
                   ------  ------
                   Actual  Actual
                  Positive Negative
```

Where:

- **TP (True Positive)**: Correctly predicted positive cases
- **TN (True Negative)**: Correctly predicted negative cases
- **FP (False Positive)**: Incorrectly predicted as positive (Type I Error)
- **FN (False Negative)**: Incorrectly predicted as negative (Type II Error)

Example: Disease Diagnosis

Scenario: COVID-19 test results for 1000 people

	Actual Positive (Has COVID)	Actual Negative (No COVID)	Total
Predicted Positive	85 (TP)	15 (FP)	100
Predicted Negative	5 (FN)	895 (TN)	900
Total	90	910	1000

Classification Performance Metrics:

1. Accuracy

Formula: `Accuracy = (TP + TN) / (TP + TN + FP + FN)`

Calculation: `(85 + 895) / 1000 = 0.98 or 98%`

Interpretation: 98% of all predictions are correct.

2. Precision (Positive Predictive Value)

Formula: `Precision = TP / (TP + FP)`

Calculation: `85 / (85 + 15) = 0.85 or 85%`

Interpretation: Of all positive predictions, 85% are actually positive. **Question:** "When the model says positive, how often is it correct?"

3. Recall (Sensitivity, True Positive Rate)

Formula: `Recall = TP / (TP + FN)`

Calculation: `85 / (85 + 5) = 0.944 or 94.4%`

Interpretation: Of all actual positive cases, 94.4% are correctly identified. **Question:** "Of all actual positives, how many did we catch?"

4. Specificity (True Negative Rate)

Formula: `Specificity = TN / (TN + FP)`

Calculation: `895 / (895 + 15) = 0.984 or 98.4%`

Interpretation: Of all actual negative cases, 98.4% are correctly identified.

5. F1-Score (Harmonic Mean of Precision and Recall)

Formula: `F1 = 2 × (Precision × Recall) / (Precision + Recall)`

Calculation: `2 × (0.85 × 0.944) / (0.85 + 0.944) = 0.895 or 89.5%`

Interpretation: Balanced measure between precision and recall.

6. False Positive Rate (1 - Specificity)

Formula: `FPR = FP / (FP + TN)`

Calculation: `15 / (15 + 895) = 0.016 or 1.6%`

Detailed Metrics Summary:

Metric	Formula	Value	Interpretation
Accuracy	(TP+TN)/(TP+TN+FP+FN)	98%	Overall correctness
Precision	TP/(TP+FP)	85%	Positive prediction reliability
Recall	TP/(TP+FN)	94.4%	Positive case detection rate
Specificity	TN/(TN+FP)	98.4%	Negative case detection rate
F1-Score	2×(P×R)/(P+R)	89.5%	Balanced precision-recall

Multiclass Confusion Matrix:

Example: Email Classification (3 classes)

	Actual Spam	Actual Personal	Actual Work	Total
Predicted Spam	45	3	2	50

	Actual Spam	Actual Personal	Actual Work	Total
Predicted Personal	5	92	3	100
Predicted Work	2	1	47	50
Total	52	96	52	200

Per-Class Metrics:

For Spam Class:

- Precision = 45/50 = 90%
- Recall = 45/52 = 86.5%
- F1-Score = 88.2%

Macro vs Micro Averages:

- **Macro Average**: Average of per-class metrics
- **Micro Average**: Global calculation across all classes

When to Use Which Metric:

Scenario	Primary Metric	Reason
Balanced Classes	Accuracy	Equal importance to all classes
Imbalanced Classes	F1-Score	Balances precision and recall
Cost of False Positives High	Precision	Minimize incorrect positives
Cost of False Negatives High	Recall	Minimize missed positives
Medical Screening	Recall	Don't miss actual cases
Spam Detection	Precision	Don't block legitimate emails

---

b) **What is loss function, explain how Gradient descent helps to Minimize the loss**

What is a Loss Function?

Definition: A loss function (also called cost function or objective function) measures how well a machine learning model performs by quantifying the difference between predicted and actual values.

Purpose:

- Measure model performance
- Guide parameter optimization
- Provide feedback during training
- Enable comparison between models

Types of Loss Functions:

1. Regression Loss Functions

Mean Squared Error (MSE):

```
MSE = (1/n) × Σᵢ₌₁ⁿ (yᵢ − ŷᵢ)²
```

Mean Absolute Error (MAE):

```
MAE = (1/n) × Σᵢ₌₁ⁿ |yᵢ − ŷᵢ|
```

2. Classification Loss Functions

Binary Cross-Entropy (Log Loss):

```
BCE = −(1/n) × Σᵢ₌₁ⁿ [yᵢ log(ŷᵢ) + (1−yᵢ) log(1−ŷᵢ)]
```

Categorical Cross-Entropy:

```
CCE = −(1/n) × Σᵢ₌₁ⁿ Σⱼ₌₁ᶜ yᵢⱼ log(ŷᵢⱼ)
```

What is Gradient Descent?

Definition: Gradient descent is an optimization algorithm that iteratively adjusts model parameters to minimize the loss function by moving in the direction of steepest decrease.

Core Concept:

- Find the minimum of a function by following its gradient (slope)
- Gradient points in direction of steepest increase
- Move in opposite direction to minimize

Mathematical Foundation:

Gradient: Vector of partial derivatives

```
∇f(θ) = [∂f/∂θ₁, ∂f/∂θ₂, ..., ∂f/∂θₙ]
```

Parameter Update Rule:

```
θₙₑw = θₒₗd − α × ∇f(θₒₗd)
```

Where:

- θ = model parameters
- α = learning rate
- $\nabla f(\theta)$ = gradient of loss function

Step-by-Step Process:

Example: Linear Regression with MSE

Model: $y = \beta_0 + \beta_1 x$ **Loss Function:** $MSE = (1/n) \times \Sigma(y_i - (\beta_0 + \beta_1 x_i))^2$

Step 1: Calculate Partial Derivatives

```
∂MSE/∂β₀ = −(2/n) × Σ(yᵢ − (β₀ + β₁xᵢ))
∂MSE/∂β₁ = −(2/n) × Σ(yᵢ − (β₀ + β₁xᵢ)) × xᵢ
```

Step 2: Update Parameters

```
β₀ₙₑw = β₀ₒₗd − α × (∂MSE/∂β₀)
β₁ₙₑw = β₁ₒₗd − α × (∂MSE/∂β₁)
```

Step 3: Repeat Until Convergence

Numerical Example:

Dataset:

x	y
1	3
2	5
3	7

Initial Parameters: $\beta_0 = 0$, $\beta_1 = 1$, $\alpha = 0.01$

Iteration 1:

```
Predictions: ŷ₁ = 1, ŷ₂ = 2, ŷ₃ = 3
Errors: e₁ = 2, e₂ = 3, e₃ = 4
MSE = (4 + 9 + 16)/3 = 9.67

Gradients:
∂MSE/∂β₀ = −(2/3) × (2 + 3 + 4) = −6
∂MSE/∂β₁ = −(2/3) × (2×1 + 3×2 + 4×3) = −16
```

```
Updates:
β₀ = 0 - 0.01 × (-6) = 0.06
β₁ = 1 - 0.01 × (-16) = 1.16
```

Iteration 2:

```
Continue until convergence...
Final: β₀ ≈ 1, β₁ ≈ 2
Model: y = 1 + 2x
```

Types of Gradient Descent:

1. Batch Gradient Descent

```
for epoch in range(num_epochs):
    gradient = compute_gradient(entire_dataset)
    parameters = parameters - learning_rate * gradient
```

Characteristics:

- Uses entire dataset for each update
- Stable convergence
- Slow for large datasets

2. Stochastic Gradient Descent (SGD)

```
for epoch in range(num_epochs):
    for sample in dataset:
        gradient = compute_gradient(single_sample)
        parameters = parameters - learning_rate * gradient
```

Characteristics:

- Uses one sample per update
- Fast updates but noisy convergence
- Good for large datasets

3. Mini-batch Gradient Descent

```
for epoch in range(num_epochs):
    for batch in batches:
```

```
        gradient = compute_gradient(batch)
        parameters = parameters - learning_rate * gradient
```

Characteristics:

- Uses small batches (e.g., 32, 64, 128 samples)
- Balance between stability and speed
- Most commonly used in practice

Learning Rate (α) Effects:

Too Large Learning Rate:

```
Loss
  ↑     *
        /|\
       / | \
      /  |  \
     *   |   * → Oscillation, no convergence
        Parameters →
```

Too Small Learning Rate:

```
Loss
  ↑   *
      |*
      | *
      |  *
      |   * → Very slow convergence
        Parameters →
```

Optimal Learning Rate:

```
Loss
  ↑   *
      |\
      | \
      |  \
      |   * → Smooth, fast convergence
        Parameters →
```

Advanced Gradient Descent Variants:

1. Momentum:

```
v_t = γv_{t-1} + α∇f(θ_t)
θ_{t+1} = θ_t - v_t
```

- Accelerates convergence
- Reduces oscillation

2. Adam (Adaptive Moment Estimation):

```
m_t = β_1 m_{t-1} + (1-β_1)∇f(θ_t)        # First moment
v_t = β_2 v_{t-1} + (1-β_2)(∇f(θ_t))^2     # Second moment
θ_{t+1} = θ_t - α(m_t/(√v_t + ε))         # Update
```

- Adaptive learning rates
- Combines benefits of momentum and RMSprop

Why Gradient Descent Works:

1. **Mathematical Guarantee:** For convex functions, guaranteed to find global minimum
2. **Efficient**: Scales well with number of parameters
3. **General Purpose**: Works with any differentiable loss function
4. **Automated**: No manual parameter tuning during optimization

Challenges and Solutions:

Challenge	Problem	Solution
Local Minima	Gets stuck in suboptimal solutions	Use momentum, random restarts
Saddle Points	Gradient ≈ 0 but not minimum	Use advanced optimizers (Adam)
Vanishing Gradients	Gradients become very small	Proper initialization, normalization
Learning Rate	Hard to choose optimal value	Adaptive learning rates, learning schedules

Question 7

(i) **Explain how linear regression works using the mathematical expression for a sample dataset? Also, what are the different performance evaluation metrics, and how do they work?**

How Linear Regression Works:

Objective: Find the best-fitting straight line through data points that minimizes the sum of squared errors.

Sample Dataset: Student Study Hours vs Exam Scores

Student	Study Hours (x)	Exam Score (y)
A	2	55
B	4	65
C	6	75
D	8	85
E	10	95

Mathematical Expression:

Linear Model: $y = \beta_0 + \beta_1 x + \varepsilon$

Where:

- y = dependent variable (exam score)
- x = independent variable (study hours)
- β_0 = y-intercept (score when study hours = 0)
- β_1 = slope (score increase per study hour)
- ε = error term

Step-by-Step Mathematical Calculation:

Step 1: Calculate Means

```
x̄ = (2 + 4 + 6 + 8 + 10) / 5 = 30 / 5 = 6 hours
ȳ = (55 + 65 + 75 + 85 + 95) / 5 = 375 / 5 = 75 points
```

Step 2: Calculate Required Sums

Student	x	y	$(x - \bar{x})$	$(y - \bar{y})$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$
A	2	55	-4	-20	80	16
B	4	65	-2	-10	20	4
C	6	75	0	0	0	0
D	8	85	2	10	20	4
E	10	95	4	20	80	16
Sum					**200**	**40**

Step 3: Calculate Slope (β_1)

```
β₁ = Σ[(xᵢ − x̄)(yᵢ − ȳ)] / Σ[(xᵢ − x̄)²]
β₁ = 200 / 40 = 5 points per hour
```

Step 4: Calculate Intercept (β_0)

```
β₀ = ȳ − β₁x̄
β₀ = 75 − 5(6) = 75 − 30 = 45 points
```

Step 5: Final Regression Equation

```
y = 45 + 5x
```

Interpretation:

- Base score (without studying): 45 points
- Each additional hour of study increases score by 5 points

Step 6: Make Predictions

```
For 3 hours: y = 45 + 5(3) = 60 points
For 7 hours: y = 45 + 5(7) = 80 points
For 12 hours: y = 45 + 5(12) = 105 points (extrapolation)
```

Matrix Form Solution:

Normal Equation: `β = (X'X)⁻¹X'y`

```
X = [1  2 ]    y = [55]
    [1  4 ]        [65]
    [1  6 ]        [75]
    [1  8 ]        [85]
    [1  10]        [95]

X'X = [5   30]    X'y = [375]
      [30  220]         [2450]

β = (X'X)⁻¹X'y = [45]  → β₀ = 45, β₁ = 5
                 [5 ]
```

Performance Evaluation Metrics:

1. Mean Squared Error (MSE)

Formula: $MSE = (1/n) \times \Sigma(y_i - \hat{y}_i)^2$

Calculation:

Student	Actual (y)	Predicted (ŷ)	Error (y - ŷ)	Squared Error
A	55	55	0	0
B	65	65	0	0
C	75	75	0	0
D	85	85	0	0
E	95	95	0	0

```
MSE = (0 + 0 + 0 + 0 + 0) / 5 = 0
```

Interpretation: Perfect fit! (Rarely happens with real data)

2. Root Mean Squared Error (RMSE)

Formula: $RMSE = \sqrt{MSE}$

```
RMSE = √0 = 0 points
```

Interpretation: Average prediction error in original units.

3. Mean Absolute Error (MAE)

Formula: $MAE = (1/n) \times \Sigma|y_i - \hat{y}_i|$

```
MAE = (0 + 0 + 0 + 0 + 0) / 5 = 0 points
```

Interpretation: Average absolute prediction error.

4. R-squared (Coefficient of Determination)

Formula: $R^2 = 1 - (SSres / SStot)$

Where:

- $SSres = \Sigma(y_i - \hat{y}_i)^2$ (Sum of Squared Residuals)
- $SStot = \Sigma(y_i - \bar{y})^2$ (Total Sum of Squares)

Calculation:

```
SSres = 0 (perfect predictions)
SStot = (−20)² + (−10)² + 0² + 10² + 20² = 400 + 100 + 0 + 100 + 400 =
1000

R² = 1 − (0 / 1000) = 1.0 or 100%
```

Interpretation: Model explains 100% of variance in the data.

5. Adjusted R-squared

Formula: `Adj R² = 1 − [(1 − R²)(n − 1) / (n − p − 1)]`

Where:

- n = number of observations
- p = number of predictors

```
Adj R² = 1 − [(1 − 1.0)(5 − 1) / (5 − 1 − 1)] = 1 − [0 × 4 / 3] = 1.0
```

Realistic Example with Noise:

Modified Dataset (with realistic errors):

Student	Study Hours (x)	Actual Score (y)	Predicted (ŷ)	Error
A	2	53	55	-2
B	4	67	65	2
C	6	74	75	-1
D	8	86	85	1
E	10	94	95	-1

Performance Metrics:

```
MSE = (4 + 4 + 1 + 1 + 1) / 5 = 2.2
RMSE = √2.2 = 1.48 points
MAE = (2 + 2 + 1 + 1 + 1) / 5 = 1.4 points
R² = 1 − (11 / 1000) = 0.989 or 98.9%
```

Metric Comparison:

Metric	Range	Units	Interpretation	Best Value
MSE	[0, ∞)	(target units)²	Penalizes large errors more	0

Metric	Range	Units	Interpretation	Best Value
RMSE	[0, ∞)	target units	Average error magnitude	0
MAE	[0, ∞)	target units	Average absolute error	0
R^2	(-∞, 1]	unitless	Proportion of variance explained	1

When to Use Each Metric:

- **MSE/RMSE**: When large errors are particularly undesirable
- **MAE**: When all errors should be treated equally
- **R^2**: When you want to understand how much variance is explained
- **Adjusted R^2**: When comparing models with different numbers of features

---

(ii) **Compare Supervised, Unsupervised, and Semi-Supervised Learning. Provide examples of when each type of learning is most suitable.**

Overview of Learning Types:

Learning Type	Labels Available	Goal	Example
Supervised	Yes (input-output pairs)	Learn mapping function	Email spam detection
Unsupervised	No (only inputs)	Find hidden patterns	Customer segmentation
Semi-Supervised	Partially (few labeled, many unlabeled)	Improve with limited labels	Web page classification

---

1. Supervised Learning

Definition: Learning from labeled training data to make predictions on new, unseen data.

Characteristics:

- Training data includes both input features and correct output labels
- Goal is to learn a mapping function: f(X) → Y
- Performance can be measured using known correct answers
- Requires significant labeled data

Types of Supervised Learning:

Classification: Predicting discrete categories

```
Input: Email text
Output: [Spam, Not Spam]
```

```
Input: Medical symptoms
Output: [Disease A, Disease B, Healthy]
```

Regression: Predicting continuous values

```
Input: House features (size, location, age)
Output: Price ($200,000)

Input: Weather conditions
Output: Temperature (25.3°C)
```

Common Algorithms:

- **Classification**: Logistic Regression, Decision Trees, Random Forest, SVM, Neural Networks
- **Regression**: Linear Regression, Polynomial Regression, Ridge, Lasso

Examples and Use Cases:

1. Medical Diagnosis

```
Training Data:
Patient 1: [symptoms] → Diabetes
Patient 2: [symptoms] → Healthy
Patient 3: [symptoms] → Hypertension
...

New Patient: [symptoms] → ?
```

2. Image Recognition

```
Training Data:
Image 1: [pixel values] → Cat
Image 2: [pixel values] → Dog
Image 3: [pixel values] → Bird
...

New Image: [pixel values] → ?
```

3. Financial Credit Scoring

```
Training Data:
Customer 1: [income, age, history] → Approved
Customer 2: [income, age, history] → Rejected
...
```

```
New Customer: [income, age, history] → ?
```

When to Use Supervised Learning:

- Abundant labeled data available
- Clear target variable defined
- Need to make predictions on new data
- Regulatory requirements for explainable decisions

Advantages:

- High accuracy when sufficient data available
- Clear performance evaluation metrics
- Well-established algorithms and techniques
- Direct applicability to business problems

Disadvantages:

- Requires expensive labeled data
- May not generalize beyond training distribution
- Limited to predefined categories/ranges
- Vulnerable to labeling errors

---

2. Unsupervised Learning

Definition: Learning patterns and structures from data without labeled examples.

Characteristics:

- Only input data available, no target labels
- Goal is to discover hidden patterns or structures
- Performance evaluation is more subjective
- Exploratory in nature

Types of Unsupervised Learning:

Clustering: Grouping similar data points

```
Input: Customer purchase data
Output: Customer segments [Budget-conscious, Premium, Occasional]
```

Association Rule Mining: Finding relationships between variables

```
Input: Market basket data
Output: "People who buy bread also buy butter" (70% confidence)
```

Dimensionality Reduction: Reducing number of features while preserving information

```
Input: 1000 features
Output: 10 most important features
```

Anomaly Detection: Identifying unusual patterns

```
Input: Network traffic data
Output: Unusual patterns that might indicate cyber attacks
```

Common Algorithms:

- **Clustering**: K-Means, Hierarchical Clustering, DBSCAN
- **Dimensionality Reduction**: PCA, t-SNE, UMAP
- **Association Rules**: Apriori, FP-Growth
- **Anomaly Detection**: Isolation Forest, One-Class SVM

Examples and Use Cases:

1. Customer Segmentation

```
E-commerce Data:
- Purchase frequency
- Average order value
- Product categories
- Time of purchases

Output Segments:
- High-value customers
- Bargain hunters
- Seasonal shoppers
- New customers
```

2. Gene Expression Analysis

```
Biological Data:
- Gene expression levels
- Protein interactions
- Metabolic pathways
```

```
Discoveries:
- Gene clusters with similar functions
- Disease-related gene patterns
- Drug target identification
```

3. Market Basket Analysis

```
Retail Transaction Data:
Transaction 1: [Bread, Butter, Milk]
Transaction 2: [Bread, Jam, Coffee]
Transaction 3: [Milk, Cookies, Tea]

Rules Discovered:
- If Bread → then Butter (confidence: 85%)
- If Coffee → then Sugar (confidence: 90%)
```

When to Use Unsupervised Learning:

- No labeled data available
- Exploratory data analysis needed
- Want to discover hidden patterns
- Data preprocessing (feature reduction)
- Market research and customer insights

Advantages:

- No need for expensive labeled data
- Can discover unexpected patterns
- Useful for data exploration
- Can handle large, complex datasets

Disadvantages:

- Difficult to evaluate results objectively
- Results may not be actionable
- Requires domain expertise for interpretation
- May find spurious patterns

---

3. Semi-Supervised Learning

Definition: Learning from a combination of labeled and unlabeled data, typically with much more unlabeled data than labeled.

Characteristics:

- Small amount of labeled data + large amount of unlabeled data
- Leverages patterns in unlabeled data to improve supervised learning

- Cost-effective when labeling is expensive
- Combines benefits of both supervised and unsupervised learning

How Semi-Supervised Learning Works:

Basic Approach:

1. Train initial model on labeled data
2. Use model to predict labels on unlabeled data
3. Add most confident predictions to training set
4. Retrain model with expanded dataset
5. Repeat until convergence

Common Algorithms:

- **Self-Training**: Use model's own predictions
- **Co-Training**: Use multiple views of the data
- **Graph-based Methods**: Exploit data manifold structure
- **Generative Models**: Model data distribution

Examples and Use Cases:

1. Web Page Classification

```
Scenario: Classify web pages into categories

Labeled Data (expensive to create):
- 1,000 web pages manually labeled
- Categories: News, Sports, Technology, Entertainment

Unlabeled Data (free):
- 100,000 web pages without labels

Process:
1. Train classifier on 1,000 labeled pages
2. Apply to 100,000 unlabeled pages
3. Select most confident predictions (e.g., >95% confidence)
4. Add these to training set
5. Retrain and repeat
```

2. Medical Image Analysis

```
Scenario: Detect cancer in X-rays

Labeled Data:
- 500 X-rays labeled by radiologists ($$$)
- Labels: Cancer, No Cancer
```

```
Unlabeled Data:
- 50,000 X-rays available in hospital database

Benefit:
- Expensive to get expert labels for all images
- Semi-supervised learning improves accuracy using unlabeled images
- Particularly useful when diseases are rare
```

3. Speech Recognition

```
Scenario: Build voice assistant for new language

Labeled Data:
- 10 hours of transcribed speech in target language
- Very expensive to transcribe more

Unlabeled Data:
- 1000 hours of untranscribed speech in target language

Process:
- Use labeled data to train initial model
- Apply model to predict transcripts for unlabeled speech
- Use confident predictions to expand training data
- Achieve performance comparable to fully supervised model with 100x more
labels
```

When to Use Semi-Supervised Learning:

- Large amounts of unlabeled data available
- Labeling is expensive or time-consuming
- Limited budget for data annotation
- Domain where unlabeled data contains useful structure
- Initial labeled dataset is small but some structure exists

Advantages:

- Improved performance over supervised learning with limited labels
- Cost-effective compared to fully supervised approach
- Can leverage abundant unlabeled data
- Particularly effective when labeled data is scarce

Disadvantages:

- More complex than supervised or unsupervised learning
- Risk of reinforcing initial model biases
- Performance depends on quality of unlabeled data
- May not work well if unlabeled data distribution differs from labeled data

Detailed Comparison:

Aspect	Supervised	Unsupervised	Semi-Supervised
Data Requirements	Large labeled dataset	Only unlabeled data	Small labeled + large unlabeled
Cost	High (labeling)	Low	Medium
Performance	High (with enough data)	Variable	Good with less labeled data
Interpretability	High	Medium	Medium
Use Case	Prediction tasks	Pattern discovery	Limited label scenarios
Evaluation	Clear metrics	Subjective	Clear metrics

Choosing the Right Approach:

Choose Supervised Learning when:

- You have abundant labeled data
- Clear prediction task is defined
- High accuracy is critical
- Regulatory compliance requires explainable models

Choose Unsupervised Learning when:

- No labeled data available
- Goal is exploration and insight discovery
- Want to understand data structure
- Preprocessing step for other algorithms

Choose Semi-Supervised Learning when:

- Limited labeled data but abundant unlabeled data
- Labeling is expensive or time-consuming
- Want to improve supervised model performance
- Domain has natural structure that can be exploited

Real-World Applications Summary:

Domain	Supervised Example	Unsupervised Example	Semi-Supervised Example
Healthcare	Disease diagnosis	Patient clustering	Drug discovery
Finance	Credit scoring	Fraud pattern discovery	Risk assessment
Technology	Spam detection	User behavior analysis	Content recommendation
Marketing	Sales forecasting	Customer segmentation	Product categorization

Domain	Supervised Example	Unsupervised Example	Semi-Supervised Example
Manufacturing	Quality control	Process optimization	Defect detection

Question 8

(i) **Explain in detail about logistic regression classifier, with suitable example**

What is Logistic Regression?

Definition: Logistic regression is a statistical method used for binary classification that models the probability of an instance belonging to a particular class using the logistic (sigmoid) function.

Key Concepts:

- Predicts probabilities (0 to 1) rather than direct classifications
- Uses maximum likelihood estimation for parameter learning
- Creates linear decision boundaries in feature space
- Extension of linear regression for classification problems

Mathematical Foundation:

1. Linear Combination (Logits)

```
z = β₀ + β₁x₁ + β₂x₂ + ... + βₙxₙ
```

2. Sigmoid Function

```
σ(z) = 1 / (1 + e^(-z))
```

3. Probability Prediction

```
P(y = 1|x) = σ(β₀ + β₁x₁ + β₂x₂ + ... + βₙxₙ)
P(y = 0|x) = 1 - P(y = 1|x)
```

4. Classification Decision

```
If P(y = 1|x) ≥ 0.5: Classify as Class 1
If P(y = 1|x) < 0.5: Classify as Class 0
```

Detailed Example: Credit Approval System

Scenario: Bank wants to automate loan approval decisions based on applicant information.

Dataset:

Applicant	Income ($000)	Credit Score	Years Employed	Approved (y)
A	30	600	2	0
B	50	700	5	1
C	80	750	8	1
D	25	550	1	0
E	60	720	6	1
F	40	650	3	0
G	70	780	7	1
H	35	580	2	0

Step-by-Step Implementation:

Step 1: Model Setup

Features:

- x_1 = Income (in $000)
- x_2 = Credit Score
- x_3 = Years Employed

Model:

```
z = β₀ + β₁(Income) + β₂(Credit_Score) + β₃(Years_Employed)
P(Approved) = 1 / (1 + e^(-z))
```

Step 2: Parameter Estimation (Using Maximum Likelihood)

Assumed Trained Parameters (simplified):

```
β₀ = -15.0
β₁ = 0.05  (coefficient for Income)
β₂ = 0.02  (coefficient for Credit Score)
β₃ = 0.3   (coefficient for Years Employed)
```

Trained Model:

```
z = -15.0 + 0.05×Income + 0.02×Credit_Score + 0.3×Years_Employed
P(Approved) = 1 / (1 + e^(-z))
```

Step 3: Making Predictions

Example 1: New Applicant

- Income: $55,000
- Credit Score: 700
- Years Employed: 4

Calculation:

```
z = -15.0 + 0.05(55) + 0.02(700) + 0.3(4)
z = -15.0 + 2.75 + 14.0 + 1.2
z = 2.95

P(Approved) = 1 / (1 + e^(-2.95))
P(Approved) = 1 / (1 + 0.052)
P(Approved) = 1 / 1.052 = 0.95 or 95%

Decision: APPROVED (since 0.95 > 0.5)
```

Example 2: Another Applicant

- Income: $30,000
- Credit Score: 580
- Years Employed: 1

Calculation:

```
z = -15.0 + 0.05(30) + 0.02(580) + 0.3(1)
z = -15.0 + 1.5 + 11.6 + 0.3
z = -1.6

P(Approved) = 1 / (1 + e^(-(-1.6)))
P(Approved) = 1 / (1 + e^1.6)
P(Approved) = 1 / (1 + 4.95)
P(Approved) = 1 / 5.95 = 0.17 or 17%

Decision: REJECTED (since 0.17 < 0.5)
```

Step 4: Model Interpretation

Coefficient Interpretation:

- **$\beta_1 = 0.05$**: Each $1,000 increase in income multiplies odds by e^0.05 = 1.051 (5.1% increase)
- **$\beta_2 = 0.02$**: Each point increase in credit score multiplies odds by e^0.02 = 1.020 (2.0% increase)
- **$\beta_3 = 0.3$**: Each additional year of employment multiplies odds by e^0.3 = 1.35 (35% increase)

Advantages of Logistic Regression:

1. **Probabilistic Output**: Provides probability estimates, not just classifications
2. **No Distribution Assumptions**: Doesn't require features to be normally distributed
3. **Fast Training**: Computationally efficient
4. **Interpretable**: Coefficients show feature importance and direction
5. **Regularization Friendly**: Works well with L1/L2 regularization
6. **Baseline Model**: Good starting point for classification problems

Limitations:

1. **Linear Decision Boundary**: Cannot handle complex non-linear relationships
2. **Sensitive to Outliers**: Extreme values can skew results
3. **Feature Independence**: Assumes features are independent
4. **Large Sample Size**: Needs adequate sample size for stable results

Decision Boundary Visualization:

For 2D case with features x_1 and x_2:

```
Decision Boundary occurs when P(y=1) = 0.5
This happens when z = 0

β₀ + β₁x₁ + β₂x₂ = 0
x₂ = −(β₀ + β₁x₁) / β₂

This is a straight line separating the two classes
```

Performance Evaluation:

Confusion Matrix for Credit Example:

	Actual Reject	Actual Approve
Predicted Reject	3	1
Predicted Approve	1	3

Metrics:

- Accuracy: (3+3)/(3+1+1+3) = 75%
- Precision: 3/(1+3) = 75%
- Recall: 3/(1+3) = 75%

Extensions:

1. Multinomial Logistic Regression: For multiple classes using softmax function:

```
P(y = k|x) = e^(z_k) / Σⱼ e^(z_j)
```

2. Ordinal Logistic Regression: For ordered categories (e.g., credit rating: Poor, Fair, Good, Excellent)

3. Regularized Logistic Regression:

- L1 (Lasso): Feature selection
- L2 (Ridge): Prevents overfitting

(ii) What is loss function, explain how Gradient descent helps to Minimize the loss

What is a Loss Function?

Definition: A loss function (also called cost function or objective function) quantifies the difference between predicted values and actual values, providing a measure of how well or poorly a model performs.

Purpose:

- Measure model performance numerically
- Guide parameter optimization during training
- Enable comparison between different models
- Provide feedback for learning algorithms

Types of Loss Functions:

For Regression Problems:

1. Mean Squared Error (MSE):

```
MSE = (1/n) Σᵢ₌₁ⁿ (yᵢ − ŷᵢ)²
```

- Penalizes large errors heavily
- Smooth and differentiable
- Sensitive to outliers

2. Mean Absolute Error (MAE):

```
MAE = (1/n) Σᵢ₌₁ⁿ |yᵢ − ŷᵢ|
```

- Treats all errors equally

- Less sensitive to outliers
- Not differentiable at zero

For Classification Problems:

3. Binary Cross-Entropy (Logistic Loss):

```
BCE = −(1/n) Σᵢ₌₁ⁿ [yᵢ log(ŷᵢ) + (1−yᵢ) log(1−ŷᵢ)]
```

- Used in logistic regression
- Penalizes confident wrong predictions heavily

4. Categorical Cross-Entropy:

```
CCE = −(1/n) Σᵢ₌₁ⁿ Σⱼ₌₁ᶜ yᵢⱼ log(ŷᵢⱼ)
```

- Multi-class classification
- One-hot encoded targets

Focus: Logistic Regression Loss Function

Log-Likelihood Function:

```
L(β) = Σᵢ₌₁ⁿ [yᵢ log(pᵢ) + (1−yᵢ) log(1−pᵢ)]

where pᵢ = 1 / (1 + e^(−(β₀ + β₁xᵢ₁ + ... + βₖxᵢₖ)))
```

Log-Loss (Negative Log-Likelihood):

```
J(β) = −L(β) = −Σᵢ₌₁ⁿ [yᵢ log(pᵢ) + (1−yᵢ) log(1−pᵢ)]
```

What is Gradient Descent?

Definition: Gradient descent is an iterative optimization algorithm that finds the minimum of a function by repeatedly moving in the direction of steepest descent (negative gradient).

Core Concept:

- Gradient points in direction of steepest increase
- Move in opposite direction to minimize function
- Update parameters proportionally to gradient magnitude

Mathematical Foundation:

Gradient: Vector of partial derivatives

```
∇J(β) = [∂J/∂β₀, ∂J/∂β₁, ∂J/∂β₂, ..., ∂J/∂βₖ]
```

Parameter Update Rule:

```
βₙₑw = βₒₗd - α × ∇J(βₒₗd)
```

Where:

- α = learning rate (step size)
- ∇J(β) = gradient of loss function

Gradient Descent for Logistic Regression:

Step 1: Calculate Partial Derivatives

For logistic regression with log-loss:

```
∂J/∂βⱼ = (1/n) Σᵢ₌₁ⁿ (pᵢ - yᵢ) × xᵢⱼ

where pᵢ = 1 / (1 + e^(-(β₀ + β₁xᵢ₁ + ... + βₖxᵢₖ)))
```

Step 2: Update Parameters

```
βⱼ = βⱼ - α × (∂J/∂βⱼ)
```

Detailed Example: Binary Classification

Dataset: Email Spam Detection

Email	Word "Free" (x_1)	Word "Money" (x_2)	Spam (y)
1	3	1	1
2	0	0	0
3	2	2	1
4	0	1	0

Model: `P(Spam) = 1 / (1 + e^(-(β₀ + β₁x₁ + β₂x₂)))`

Iteration-by-Iteration Process:

Initial Parameters: $\beta_0 = 0$, $\beta_1 = 0$, $\beta_2 = 0$, $\alpha = 0.1$

Iteration 1:

Forward Pass (Calculate Predictions):

```
For all samples: z = 0 + 0×x₁ + 0×x₂ = 0
p = 1/(1 + e⁰) = 0.5 for all samples
```

Calculate Loss:

```
J = -(1/4)[1×log(0.5) + 0×log(0.5) + 1×log(0.5) + 0×log(0.5) +
           0×log(0.5) + 1×log(0.5) + 0×log(0.5) + 1×log(0.5)]
J = -(1/4)[2×log(0.5)] = -(1/4)[2×(-0.693)] = 0.693
```

Calculate Gradients:

```
∂J/∂β₀ = (1/4)[(0.5-1) + (0.5-0) + (0.5-1) + (0.5-0)] = 0
∂J/∂β₁ = (1/4)[(0.5-1)×3 + (0.5-0)×0 + (0.5-1)×2 + (0.5-0)×0] = -0.625
∂J/∂β₂ = (1/4)[(0.5-1)×1 + (0.5-0)×0 + (0.5-1)×2 + (0.5-0)×1] = -0.375
```

Update Parameters:

```
β₀ = 0 - 0.1×0 = 0
β₁ = 0 - 0.1×(-0.625) = 0.0625
β₂ = 0 - 0.1×(-0.375) = 0.0375
```

Iteration 2:

Forward Pass:

```
Sample 1: z = 0 + 0.0625×3 + 0.0375×1 = 0.225, p = 0.556
Sample 2: z = 0 + 0.0625×0 + 0.0375×0 = 0, p = 0.5
Sample 3: z = 0 + 0.0625×2 + 0.0375×2 = 0.2, p = 0.550
Sample 4: z = 0 + 0.0625×0 + 0.0375×1 = 0.0375, p = 0.509
```

Calculate New Loss:

```
J = -(1/4)[1×log(0.556) + 0×log(0.444) + 1×log(0.550) + 0×log(0.491)]
J ≈ 0.672 (Loss decreased!)
```

Continue until convergence...

Types of Gradient Descent:

1. Batch Gradient Descent

```
for epoch in range(num_epochs):
    # Use entire dataset
    gradient = compute_gradient(X, y, parameters)
    parameters = parameters - learning_rate * gradient
```

Characteristics:

- Uses entire dataset for each update
- Stable convergence
- Computationally expensive for large datasets

2. Stochastic Gradient Descent (SGD)

```
for epoch in range(num_epochs):
    for i in range(n_samples):
        # Use single sample
        gradient = compute_gradient(X[i], y[i], parameters)
        parameters = parameters - learning_rate * gradient
```

Characteristics:

- Uses one sample per update
- Fast updates but noisy convergence
- Good for large datasets

3. Mini-batch Gradient Descent

```
for epoch in range(num_epochs):
    for batch in create_batches(X, y, batch_size):
        # Use small batch (e.g., 32, 64, 128 samples)
        gradient = compute_gradient(batch_X, batch_y, parameters)
        parameters = parameters - learning_rate * gradient
```

Characteristics:

- Balance between stability and speed
- Most commonly used in practice
- Batch sizes typically 32-512

Learning Rate Effects:

Too Large (α too big):

```
Loss    *
   ↑   /|\
       / | \
      *  |  *  → Oscillation, overshooting minimum
        Parameters →
```

Too Small (α too small):

```
Loss *
   ↑ |*
     | *
     |  *  → Very slow convergence
        Parameters →
```

Just Right:

```
Loss *
   ↑ |\
     | \
     |  \
     |   * → Smooth convergence to minimum
        Parameters →
```

Advanced Optimization Techniques:

1. Momentum:

```
v = γ×v + α×gradient
parameters = parameters − v
```

- Accelerates convergence
- Reduces oscillation
- γ typically 0.9

2. Adam (Adaptive Moment Estimation):

```
m = β₁×m + (1−β₁)×gradient        # First moment
v = β₂×v + (1−β₂)×gradient²       # Second moment
parameters = parameters − α×m/√v   # Update
```

- Adaptive learning rates
- Combines momentum and RMSprop
- β_1=0.9, β_2=0.999 typically

Why Gradient Descent Works:

1. **Mathematical Guarantee:** For convex functions, guaranteed to find global minimum
2. **Scalable:** Works efficiently with millions of parameters
3. **General Purpose:** Applicable to any differentiable loss function
4. **Automated:** No manual parameter adjustments during optimization

Convergence Criteria:

Stop when:

1. **Gradient magnitude small:** $||\nabla J|| < \varepsilon$ (e.g., $\varepsilon = 10^{-6}$)
2. **Parameter change small:** $||\beta_{new} - \beta_{old}|| < \varepsilon$
3. **Loss change small:** $|J_{new} - J_{old}| < \varepsilon$
4. **Maximum iterations reached:** Prevent infinite loops

Practical Considerations:

Feature Scaling:

```
# Standardize features to have mean=0, std=1
X_scaled = (X - mean) / std
```

- Ensures all features contribute equally
- Speeds up convergence
- Prevents numerical instabilities

Learning Rate Scheduling:

```
α(t) = α₀ / (1 + decay_rate × t)
```

- Start with larger learning rate
- Decrease as training progresses
- Fine-tune near optimum

Question 9

(i) **Describe the K-fold Cross-Validation process. How does it work, and why is it more robust than a simple train-test split? Provide a scenario where K-fold CV would be useful.**

K-fold Cross-Validation Process:

Definition: K-fold cross-validation is a resampling technique that divides the dataset into K equal-sized subsets (folds), uses K-1 folds for training and 1 fold for validation, and repeats this process K times.

Step-by-Step Process:

Step 1: Data Partitioning

- Divide the dataset into K equal-sized folds
- Each fold should be approximately the same size
- Maintain random distribution across folds

Step 2: Iterative Training and Validation

```
For i = 1 to K:
    Fold i → Validation Set
    Remaining (K-1) folds → Training Set
    Train model on training set
    Evaluate model on validation set
    Store performance metric (accuracy, F1-score, etc.)
```

Step 3: Final Performance Calculation

```
Final Performance = (1/K) × Σᵢ₌₁ᴷ Performance_i
Standard Error = √[(1/K) × Σᵢ₌₁ᴷ (Performance_i - Mean)²]
```

Visual Representation (5-Fold CV):

```
Original Dataset (1000 samples):
┌─────────────────────────────────────────────────────────┐
│  Fold 1  │  Fold 2  │  Fold 3  │  Fold 4  │  Fold 5  │
│  (200)   │  (200)   │  (200)   │  (200)   │  (200)   │
└─────────────────────────────────────────────────────────┘

Iteration 1: [VALID] │ [TRAIN] │ [TRAIN] │ [TRAIN] │ [TRAIN] │
Iteration 2: [TRAIN] │ [VALID] │ [TRAIN] │ [TRAIN] │ [TRAIN] │
Iteration 3: [TRAIN] │ [TRAIN] │ [VALID] │ [TRAIN] │ [TRAIN] │
Iteration 4: [TRAIN] │ [TRAIN] │ [TRAIN] │ [VALID] │ [TRAIN] │
Iteration 5: [TRAIN] │ [TRAIN] │ [TRAIN] │ [TRAIN] │ [VALID] │

Results: [0.85, 0.87, 0.83, 0.86, 0.84]
Mean: 0.85 ± 0.016 (85% ± 1.6%)
```

Why K-fold CV is More Robust than Simple Train-Test Split:

1. Better Use of Data

Train-Test Split:

```
1000 samples → 700 training, 300 testing
- Only 70% data used for training
- 30% never contributes to model learning
- Single evaluation point
```

K-fold CV (K=5):

```
1000 samples → 800 training, 200 testing (per fold)
- 80% data used for training in each iteration
- Every sample used for both training and validation
- 5 evaluation points averaged
```

2. Reduced Variance in Performance Estimates

Train-Test Split Issue:

```
Split 1: Train[1-700], Test[701-1000] → Accuracy: 82%
Split 2: Train[1-700], Test[301-600]  → Accuracy: 88%
Split 3: Train[301-1000], Test[1-300] → Accuracy: 79%

Problem: High variance depending on which samples end up in test set
```

K-fold CV Solution:

```
Fold 1: 85%
Fold 2: 87%
Fold 3: 83%
Fold 4: 86%
Fold 5: 84%

Mean: 85% ± 1.6% (More reliable estimate with confidence interval)
```

3. Detection of Overfitting

Train-Test Split:

- Single train/test combination might not reveal overfitting
- Lucky/unlucky splits can mask or exaggerate problems

K-fold CV:

- Multiple train/validation combinations
- High variance across folds indicates overfitting
- Consistent performance across folds indicates good generalization

4. Model Selection Reliability

Scenario: Comparing 3 models

Train-Test Split:

```
Model A: 85% (single measurement)
Model B: 87% (single measurement)
Model C: 84% (single measurement)

Choice: Model B (but is this reliable?)
```

K-fold CV:

```
Model A: 85% ± 2.1%  (83%–87% range)
Model B: 87% ± 4.2%  (83%–91% range)
Model C: 84% ± 1.8%  (82%–86% range)

Analysis: Model B has higher mean but also higher variance
Choice: Depends on risk tolerance
```

Practical Scenario: Medical Diagnosis System

Context: Developing a machine learning model to diagnose rare disease from symptoms

Dataset Characteristics:

- 500 patient records available
- Disease prevalence: 20% (100 positive, 400 negative)
- Limited data due to disease rarity
- High cost of misdiagnosis

Why K-fold CV is Essential Here:

Problem with Train-Test Split:

```
Train: 350 patients (70 positive, 280 negative)
Test: 150 patients (30 positive, 120 negative)

Issues:
1. Only 70 positive cases for training (very limited)
2. Single test evaluation (30 positive cases)
```

```
3. Results might not generalize due to small test set
4. Can't afford to "waste" 150 samples for testing only
```

K-fold CV Solution (K=5):

```
Each fold: 100 patients (20 positive, 80 negative)
Training per fold: 400 patients (80 positive, 320 negative)

Benefits:
1. 80 positive cases for training (vs 70 in train-test)
2. More robust evaluation across 5 different test sets
3. Every patient contributes to both training and validation
4. Better understanding of model stability
5. More reliable performance estimate for clinical deployment
```

Implementation Example:

```python
# Pseudo-code for K-fold CV
def k_fold_cross_validation(X, y, model, k=5):
    fold_size = len(X) // k
    scores = []

    for i in range(k):
        # Create validation fold
        val_start = i * fold_size
        val_end = (i + 1) * fold_size

        X_val = X[val_start:val_end]
        y_val = y[val_start:val_end]

        # Create training folds
        X_train = X[:val_start] + X[val_end:]
        y_train = y[:val_start] + y[val_end:]

        # Train and evaluate
        model.fit(X_train, y_train)
        score = model.evaluate(X_val, y_val)
        scores.append(score)

    return np.mean(scores), np.std(scores)

# Usage
mean_accuracy, std_accuracy = k_fold_cross_validation(X, y, model, k=5)
print(f"Accuracy: {mean_accuracy:.3f} ± {std_accuracy:.3f}")
```

Results Interpretation:

Medical Diagnosis Results:

```
Fold 1: Sensitivity=0.90, Specificity=0.85
Fold 2: Sensitivity=0.85, Specificity=0.88
Fold 3: Sensitivity=0.95, Specificity=0.82
Fold 4: Sensitivity=0.88, Specificity=0.87
Fold 5: Sensitivity=0.92, Specificity=0.86

Mean Sensitivity: 0.90 ± 0.04 (90% ± 4%)
Mean Specificity: 0.86 ± 0.02 (86% ± 2%)

Clinical Decision:
- Model consistently detects 90% of actual cases
- Model correctly identifies 86% of healthy patients
- Low variance indicates stable performance
- Ready for clinical trial
```

Choosing K Value:

K Value	Pros	Cons	When to Use
K=5	Good balance, computationally efficient	Moderate bias	Most common choice
K=10	Lower bias, good for larger datasets	Higher computational cost	Standard practice
K=n (LOOCV)	Lowest bias, maximum data usage	Very expensive, high variance	Small datasets (<100)
K=3	Fast computation	Higher bias	Large datasets, quick evaluation

When K-fold CV is Most Useful:

1. **Small to Medium Datasets** (<10,000 samples)
2. **Model Comparison and Selection**
3. **Hyperparameter Tuning**
4. **Performance Estimation for Deployment**
5. **Research and Academic Studies**
6. **High-Stakes Applications** (medical, financial)

---

a) **What are the different performance metrics of a Classification model? Explain the concept of confusion matrix and compute the below measure with definition:**

N=195	No	Yes
No	65	43

N=195	No	Yes
Yes	32	55

Compute the following metrics:

- Accuracy
- Precision
- Recall
- F1-score

Understanding the Confusion Matrix:

Definition: A confusion matrix is a table that describes the performance of a classification model by showing the actual vs predicted classifications.

Given Confusion Matrix Analysis:

```
                    Actual
                 No      Yes    Total
Predicted   No   65      43     108
           Yes   32      55      87
           Total 97     98     195
```

Matrix Components:

- **True Negative (TN)**: 65 (Correctly predicted "No")
- **False Positive (FP)**: 32 (Incorrectly predicted "Yes")
- **False Negative (FN)**: 43 (Incorrectly predicted "No")
- **True Positive (TP)**: 55 (Correctly predicted "Yes")

Verification: TN + FP + FN + TP = 65 + 32 + 43 + 55 = 195 ✓

Performance Metrics Calculations:

1. Accuracy

Definition: The proportion of correct predictions (both true positives and true negatives) among the total number of cases examined.

Formula:

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

Calculation:

```
Accuracy = (55 + 65) / (55 + 65 + 32 + 43)
Accuracy = 120 / 195
Accuracy = 0.615 or 61.5%
```

Interpretation: The model correctly classifies 61.5% of all instances.

2. Precision (Positive Predictive Value)

Definition: The proportion of positive identifications that were actually correct. It answers: "Of all instances predicted as positive, how many are actually positive?"

Formula:

```
Precision = TP / (TP + FP)
```

Calculation:

```
Precision = 55 / (55 + 32)
Precision = 55 / 87
Precision = 0.632 or 63.2%
```

Interpretation: When the model predicts "Yes", it is correct 63.2% of the time.

3. Recall (Sensitivity, True Positive Rate)

Definition: The proportion of actual positive cases that were correctly identified. It answers: "Of all actual positive instances, how many did we correctly identify?"

Formula:

```
Recall = TP / (TP + FN)
```

Calculation:

```
Recall = 55 / (55 + 43)
Recall = 55 / 98
Recall = 0.561 or 56.1%
```

Interpretation: The model correctly identifies 56.1% of all actual positive cases.

4. F1-Score

Definition: The harmonic mean of precision and recall. It provides a single metric that balances both precision and recall.

Formula:

```
F1-Score = 2 × (Precision × Recall) / (Precision + Recall)
```

Calculation:

```
F1-Score = 2 × (0.632 × 0.561) / (0.632 + 0.561)
F1-Score = 2 × 0.355 / 1.193
F1-Score = 0.710 / 1.193
F1-Score = 0.595 or 59.5%
```

Interpretation: The F1-score of 59.5% indicates moderate performance balancing precision and recall.

Additional Classification Metrics:

5. Specificity (True Negative Rate)

Definition: The proportion of actual negative cases that were correctly identified.

Formula:

```
Specificity = TN / (TN + FP)
```

Calculation:

```
Specificity = 65 / (65 + 32) = 65 / 97 = 0.670 or 67.0%
```

6. False Positive Rate

Definition: The proportion of actual negative cases that were incorrectly classified as positive.

Formula:

```
False Positive Rate = FP / (FP + TN) = 1 - Specificity
```

Calculation:

```
FPR = 32 / (32 + 65) = 32 / 97 = 0.330 or 33.0%
```

Summary Table:

Metric	Formula	Value	Interpretation
Accuracy	(TP+TN)/(TP+TN+FP+FN)	61.5%	Overall correctness
Precision	TP/(TP+FP)	63.2%	Positive prediction reliability
Recall	TP/(TP+FN)	56.1%	Positive case detection rate
F1-Score	2×(P×R)/(P+R)	59.5%	Balanced precision-recall
Specificity	TN/(TN+FP)	67.0%	Negative case detection rate

Performance Analysis:

Class-wise Performance:

"Yes" Class (Positive):

- Precision: 63.2% - When predicting "Yes", correct 63.2% of the time
- Recall: 56.1% - Catches 56.1% of actual "Yes" cases
- F1-Score: 59.5% - Moderate balanced performance

"No" Class (Negative):

- Precision: 65/(65+43) = 60.2% - When predicting "No", correct 60.2% of the time
- Recall: 65/(65+32) = 67.0% - Catches 67.0% of actual "No" cases
- F1-Score: 2×(0.602×0.670)/(0.602+0.670) = 63.4%

Model Assessment:

Strengths:

1. **Balanced Performance**: Similar performance on both classes
2. **Reasonable Specificity**: 67% of actual negatives correctly identified
3. **Moderate Precision**: 63.2% confidence in positive predictions

Weaknesses:

1. **Low Overall Accuracy**: 61.5% is below typical deployment threshold
2. **Poor Recall**: Missing 43.9% of actual positive cases
3. **High Error Rates**: 38.5% overall error rate

Error Analysis:

Type I Errors (False Positives): 32

- Impact: 32 instances incorrectly classified as "Yes"
- Business Impact: Depends on cost of false alarms

Type II Errors (False Negatives): 43

- Impact: 43 instances incorrectly classified as "No"
- Business Impact: Depends on cost of missed detections

Recommendations for Improvement:

1. **Collect More Data**: Low performance might indicate insufficient training data
2. **Feature Engineering**: Add more discriminative features
3. **Algorithm Selection**: Try different classification algorithms
4. **Threshold Tuning**: Adjust classification threshold to optimize precision/recall trade-off
5. **Class Balancing**: Address any class imbalance issues

When to Use Each Metric:

Scenario	Primary Metric	Reason
Balanced Classes	Accuracy	Equal importance to all classes
Imbalanced Classes	F1-Score	Balances precision and recall
Spam Detection	Precision	Minimize false positives (blocking legitimate emails)
Medical Screening	Recall	Minimize false negatives (missing diseases)
Quality Control	Specificity	Correctly identify non-defective products

---

Summary

This document contains 9 important questions covering key machine learning topics including:

- Cross-validation techniques
- Classification performance metrics
- Linear and logistic regression
- Overfitting and underfitting
- Regularization
- Loss functions and gradient descent
- Types of machine learning (supervised, unsupervised, semi-supervised)